

LISTA DE EXERCÍCIOS 3.8

1. Qual é o papel do bit de válido-inválido na tabela de páginas?

Resposta: o bit de válido-inválido é um campo na tabela de páginas que indica se uma dada página está residente em memória ou não. Caso a página esteja marcada como ‘i’ ocorre uma falha de página (*page fault*). Vale observar também que, quando da carga inicial de um processo em memória, inicialmente o bit de válido-inválido é ajustado para ‘i’ em todas as entradas.

2. Descreva os passos necessários para tratar uma *page fault*.

Resposta: os passos necessários para se tratar uma *page fault* são os seguintes:

- 1) SO verifica se o endereço virtual referenciado está nos limites para o processo (reg. limite) para decidir se a referência é inválida (aborte a operação) ou se a página simplesmente não está na memória (bit válido-inválido = ‘i’);
 - 2) Encontre um *frame* livre;
 - 3) Faça o *swap* da página para o frame via operação de disco agendada (*scheduled*);
 - 4) Resete tabelas para indicar que a página está em memória agora – ajuste o bit de válido = ‘v’;
 - 5) Reinicie a instrução que causou a falha de página.
3. O que vem a ser *pure demand paging*? Qual é o principal problema dessa estratégia? Como esse problema pode ser minimizado?

Resposta: refere-se a carga de um processo SEM a introdução de páginas em memória. Apenas as páginas referenciadas serão carregadas. Devido a isso, durante o funcionamento inicial do processo múltiplas *page faults* ocorrem. Isso pode ser minimizado graças à localidade de referência. Essa característica se refere a carga natural de páginas dependentes do contexto de execução corrente do processo (i.e. a linha de execução ativa no momento – ex. função ou *thread*).

4. Considere um processo cujo tempo de acesso a memória (MAT) seja de 100 nanosegundos e o tempo médio de resolução de *page-faults* (PFST) seja de 10 milisegundos. Se um acesso em 10.000 causa *page fault*, calcule o tempo de acesso efetivo (EAT) para este processo.

Resposta:

$$\begin{aligned}\text{EAT} &= (1 - p) \times 100 \times 10^{-9} + p (10 \times 10^{-3}) \\ &= 100 \times 10^{-9} - 100 \times 10^{-9} \times p + 10 \times 10^{-3} \times p \\ &= 0,1 \times 10^{-6} - 0,1 \times 10^{-6} \times p + 10 \times 10^{-3} \times p \\ &= 0,1 \times 10^{-6} - 0,1 \times 10^{-6} \times (10^{-4}) + 10 \times 10^{-3} \times (10^{-4}) \\ &= 0,1 \times 10^{-6} - 0,00001 \times 10^{-6} + 10^{-6} \\ &= 1,09999 \times 10^{-6} \\ &= \mathbf{1,1 \times 10^{-6} \text{ segs}}\end{aligned}$$

5. O que vem a ser o procedimento de *Copy-on-Write* (COW)?

Resposta: COW permite que processos pais e filhos, no ato da criação de um processo filho, compartilhem as mesmas páginas. Se qualquer um dos processos modifica a página compartilhada, somente nesse momento, a página é copiada e modificada.

6. Qual é o contexto de aplicação de políticas de substituição de páginas em *demand paging*?

Resposta: em *demand paging*, quando necessário páginas livres são alocadas a partir de um *pool* de páginas para demanda, as quais estão vazias (*zero-fill-on-demand pages*). Idealmente, o *pool* deve sempre conter *frames* vazios para atendimento rápido a demanda de página. Com isso, não seria necessário um *frame*, assim como ocorre em outros processamentos envolvendo *page fault*. Caso a memória venha a se exaurir (i.e. caso não haja *frame livre*), é necessário aplicar políticas de substituição de páginas com o intuito de se identificar uma página em memória que não esteja realmente em uso e removê-la. O ideal seria um algoritmo que resulte em um número mínimo de *page faults*.

7. Como o *modify/dirty* bit auxilia na otimização de políticas de substituição de páginas?

Resposta: o *dirty* bit reduz a sobrecarga quando da substituição de páginas. Isso ocorre porque, caso a página não tenha sido modificada, não há necessidade de escrevê-la na área de *swap*. Essa escrita só ocorrerá a critério de atualização da página sendo substituída. Com isso, evita-se o aumento do EAT, quando possível.